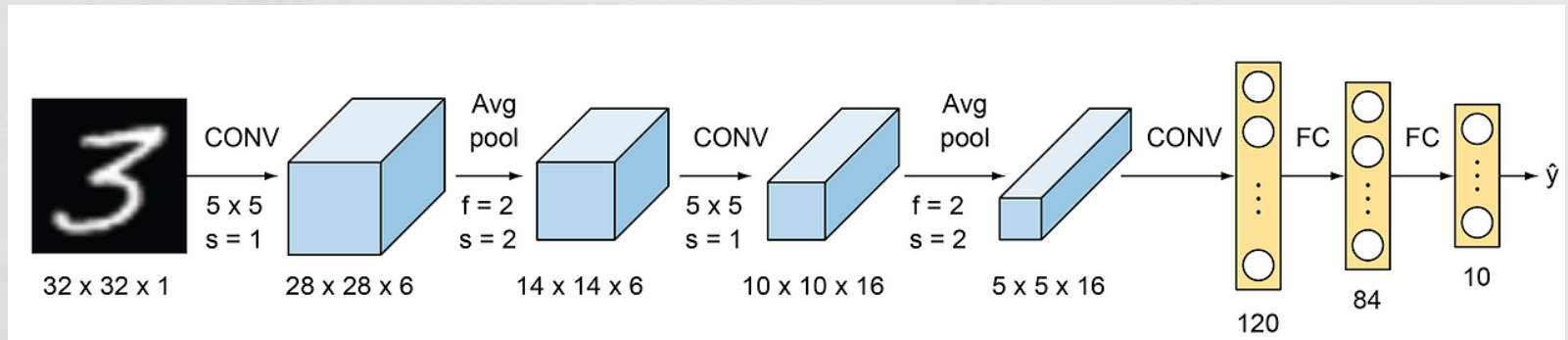


CNN 모델 종류

LeNet

■ 개요

- 최근에는 사용되지 않지만, 기본적인 CNN 구조를 가장 처음 제안한 모델이라는 점에서 의의가 있음
- 세 번의 합성곱 필터와 2번의 average pooling을 사용한 구조이며, tanh 활성화 함수와 average pooling을 사용했는데 최근에는 이 방식을 사용하지 않는다.
- 첫 번째 activation map의 depth=6 이므로, 6개의 필터를 적용했다는 것을 알 수 있다.



LeNet

```
class LeNet :
```

```
    @staticmethod
```

```
    def build(input_shape, classes):
```

```
        model = Sequential()
```

```
        model.add(Conv2D(20, kernel_size = 5, padding="same", input_shape=input_shape))
```

```
        model.add(Activation("relu"))
```

```
        model.add(MaxPooling2D(pool_size = (2,2), strides = (2,2)))
```

```
        model.add(Conv2D(50, kernel_size = 5, padding="same"))
```

```
        model.add(Activation("relu"))
```

```
        model.add(MaxPooling2D(pool_size = (2,2), strides = (2,2)))
```

```
        model.add(Flatten())
```

```
        model.add(Dense(500))
```

```
        model.add(Activation("relu"))
```

```
        model.add(Dense(10))
```

```
        model.add(Activation("softmax"))
```

```
        return model
```

```
model = Sequential() model.add(Conv2D(filters=32, kernel_size=(5,5), padding='same', activation='relu', input_shape=(28, 28, 1))) model.add(MaxPool2D(strides=2
```

LeNet

```
model = Sequential()
```

```
model.add(Conv2D(filters=32, kernel_size=(5,5), padding='same',  
activation='relu', input_shape=(28, 28, 1)))
```

```
model.add(MaxPool2D(strides=2))
```

```
model.add(Conv2D(filters=48, kernel_size=(5,5), padding='valid',  
activation='relu'))
```

```
model.add(MaxPool2D(strides=2))
```

```
model.add(Flatten())
```

```
model.add(Dense(256, activation='relu'))
```

```
model.add(Dense(84, activation='relu'))
```

```
model.add(Dense(10, activation='softmax'))
```

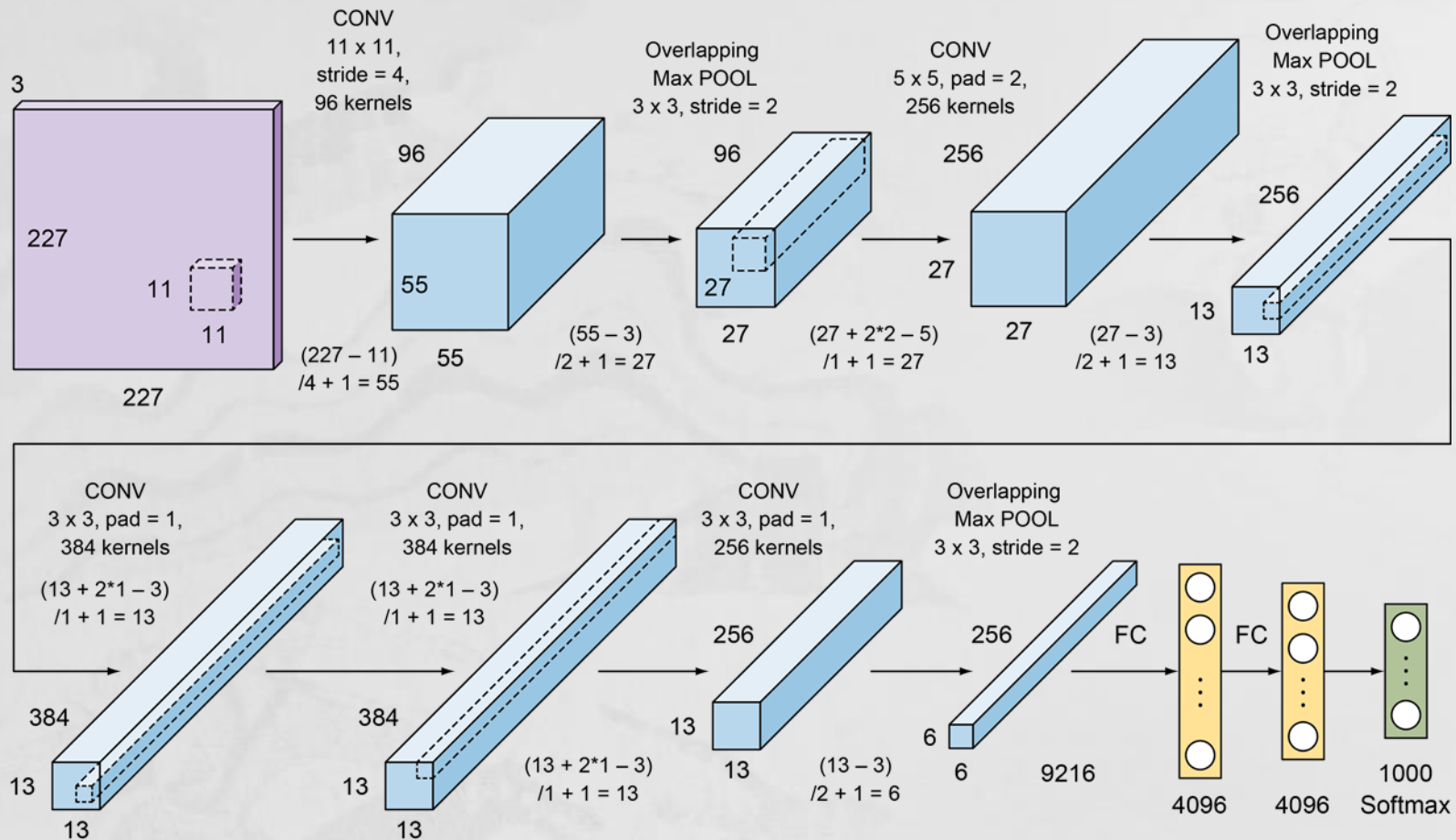
AlexNet

■ 개요

- 1998년 LeNet이 제안된 이후 CNN 알고리즘의 뚜렷한 성능 개선이 이루어지지 않았는데, 이러한 CNN 침체기를 깨면서 2012년 이미지넷 경진대회에서 우승한 모델
- 비교적 구조가 간단해서 최근에는 많이 사용되지 않음
- LeNet과 유사하게 여러 개의 합성곱 필터와 풀링 레이어를 가지고 있다는 것은 동일하지만, 활성화 함수로 relu로 사용하고 max pooling을 사용
- 5개의 합성곱 층과 3번의 풀링을 사용함.

AlexNet

■ 개요



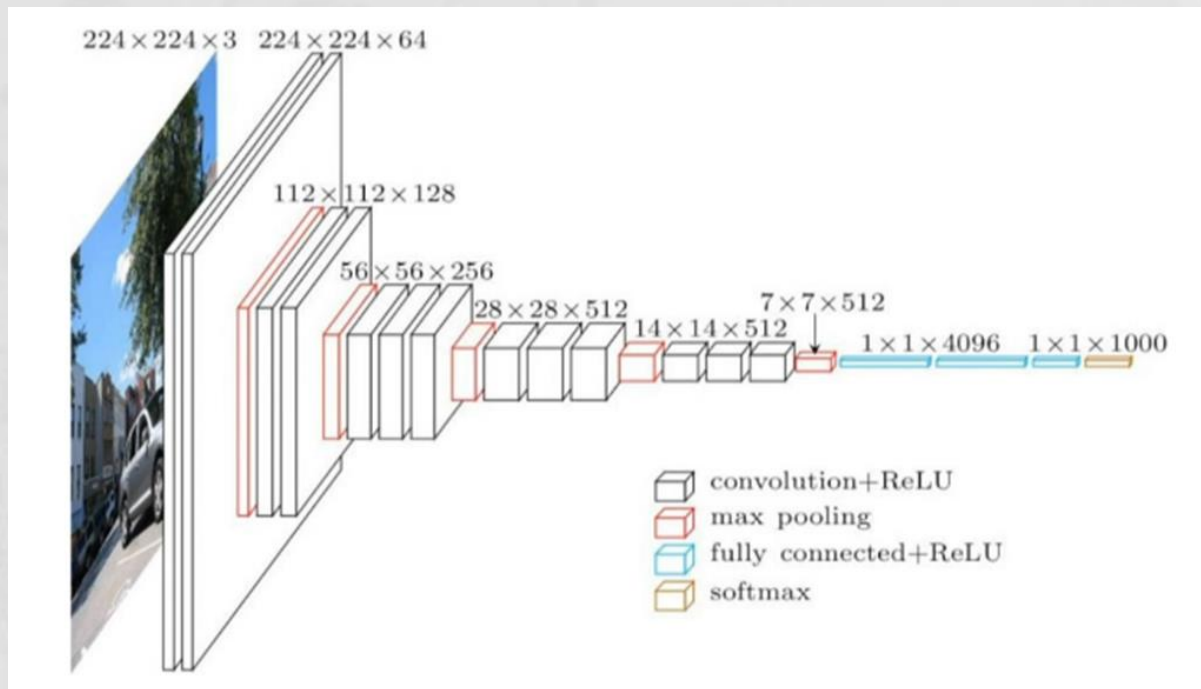
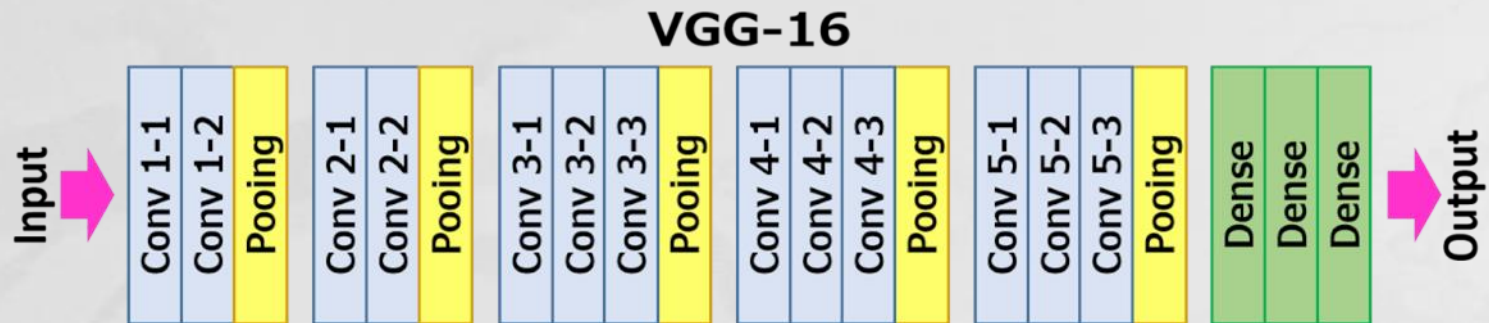
VGGNet

■ 개요

- 2014년 옥스포드 VGG그룹에서 제안된 모델로 이미지넷 대회에서 준우승한 모델
- 전체적인 구조는 AlexNet과 유사하지만 3x3필터를 stride=1로 설정해서 반복해서 적용
- pooling layer를 제외하고 총 16개 층이어서 VGG-16이라고 부른다. pooling layer에는 2x2로 max pooling을 stride=2로 해서 적용
- VGG-16 과 VGG-19가 있는데 주로 16 버전이 사용된다. 같은 년도 대회에서 우승 모델은 Inception Net이지만 VGG-16이 비교적 구조가 간단하고 사용이 편리해서 더 많이 사용(ResNet 다음으로 많이 사용됨).

VGGNet

■ VGGNet



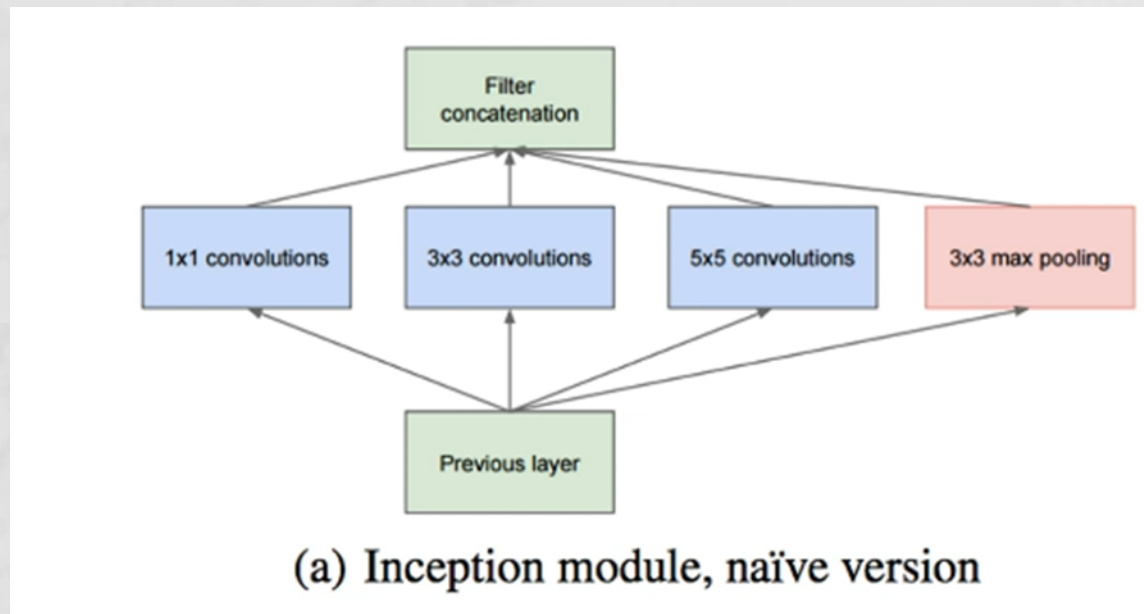
VGGNet

```
def VGG16():
    model = keras.Sequential()
    # 첫 번째 Conv Block
    # 입력 Shape는 ImageNet 데이터 세트의 크기와 같은 RGB 영상 (224 x 224 x 3)입니다.
    model.add(keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = "same", activation="relu", input_shape = (224, 224, 3)))
    model.add(keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = "same", activation = "relu", ))
    model.add(keras.layers.MaxPooling2D(2))
    # 두 번째 Conv Block
    model.add(keras.layers.Conv2D(filters = 128, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 128, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.MaxPooling2D(2)),
    # 세 번째 Conv Block
    model.add(keras.layers.Conv2D(filters = 256, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 256, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 256, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.MaxPooling2D(2))
    # 네 번째 Conv Block
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.MaxPooling2D(2)),
    # 다섯 번째 Conv Block
    # TODO: 다섯 번째 Conv Block은 네 번째 Block과 동일합니다. 앞의 코드를 참고하여 다섯 번째 Block를 추가하세요
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.Conv2D(filters = 512, kernel_size = (3,3), activation = "relu", padding = "same" ))
    model.add(keras.layers.MaxPooling2D(2)),
    # Fully Connected Layer
    model.add(keras.layers.Flatten()),
    model.add(keras.layers.Dense(4096, activation = "relu")),
    model.add(keras.layers.Dense(4096, activation = "relu")),
    model.add(keras.layers.Dense(1000, activation = "softmax")) # 마지막 레이어의 노드수를 2로 변경하세요
    return model
```

InceptionNet (=Google Net)

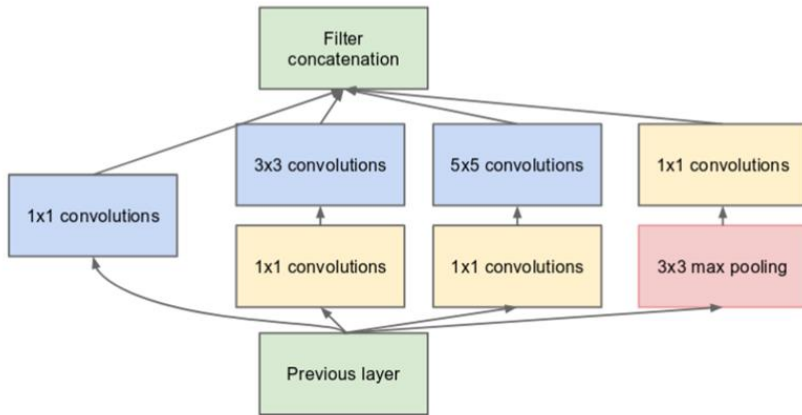
■ 개요

- 구글에서 제안한 2014년 이미지넷 경진대회 우승 모델
- 인셉션 모듈이라는 방식을 사용해서 모델 성능을 개선시킴. 인셉션 모듈은 이후 모델에서 지속적으로 사용

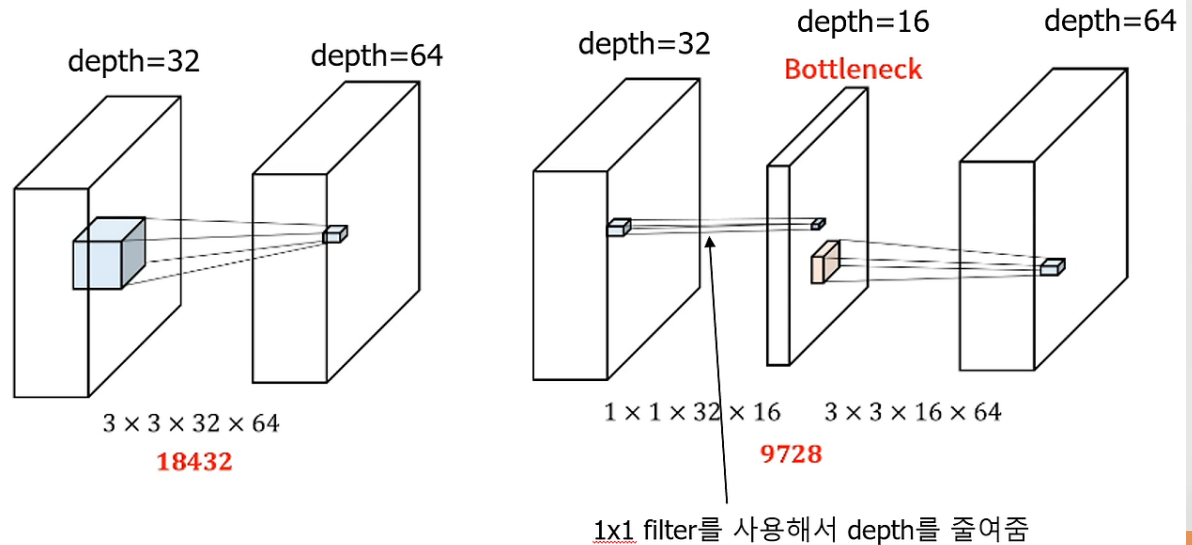


InceptionNet (= Google Net)

■ 개요

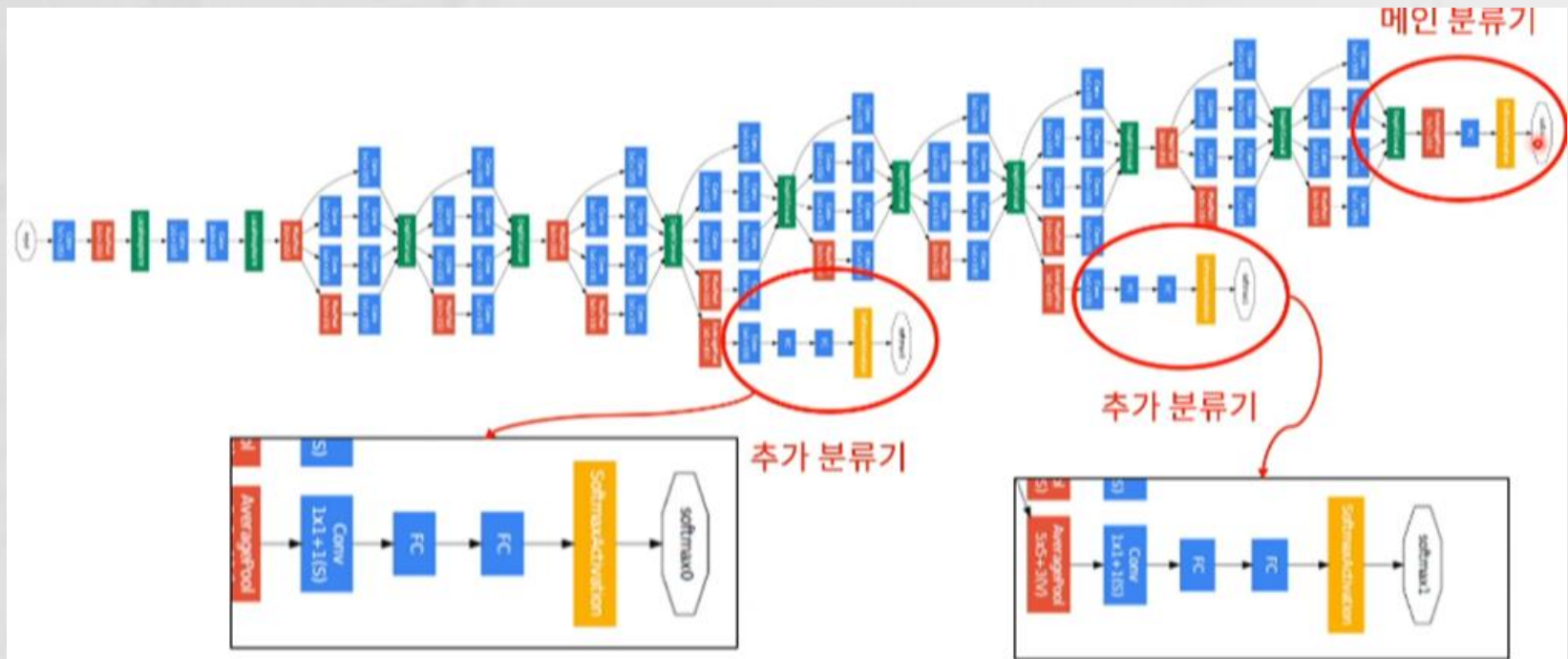


(b) Inception module with dimension reductions



InceptionNet (= Google Net)

■ 개요



ResNet (Residual Network)

■ 개요

- 마이크로소프트에서 제안하고 2015년 이미지넷에서 우승한 모델. 당시 최초로 사람의 분류 성능을 뛰어넘으면서 인공지능에 대한 기대가 증가하게 되는 계기가 되었다. 최근까지 가장 많이 사용되는 사전학습 모델
- 좌측은 VGG-19 모델, 중간은 레이어가 34개로 더 깊어진 구조, 우측이 34개의 레이어를 가진 residual net 구조

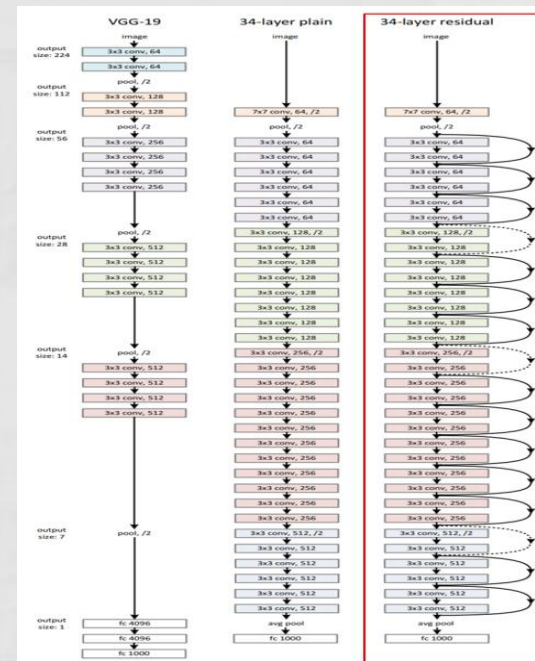


Figure 3. Example network architectures for ImageNet. Left: the

ResNet (Residual Network)

```
import tensorflow as tf

from tensorflow.keras.layers import Conv2D, BatchNormalization, Activation, Add

def residual_block(input_tensor, filters):
    x = Conv2D(filters, (3, 3), padding='same')(input_tensor)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Conv2D(filters, (3, 3), padding='same')(x)
    x = BatchNormalization()(x)
    x = Add()([x, input_tensor])
    x = Activation('relu')(x)
    return x

def create_resnet(input_shape, num_classes):
    inputs = tf.keras.Input(shape=input_shape)
    x = Conv2D(64, (7, 7), strides=(2, 2), padding='same')(inputs)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = residual_block(x, 64)
    x = residual_block(x, 64)
    x = residual_block(x, 64)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)
    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    return model
```


MobileNet

■ 개요

- 모바일과 같이 컴퓨팅 파워가 부족한 기기에서 이미지 분류를 하기 위해서 개발된 모델
- Depthwise Separable Convolution 구조를 사용함.
- depth=5인 activation map 의 채널(depth)별로 depth=1인 필터를 적용한 모델. 그리고 최종적으로 인셉션 넷과 동일하게 마지막에 1x1 필터를 적용하는 보틀넥 구조를 사용함.
- 성능은 크게 감소시키지 않으면서 파라미터 수를 감소시켜서 빠르게 학습이 가능

MobileNet

```
from tensorflow.keras.layers import *
from tensorflow.keras import *

class Mobilenet_V1():
    def __init__(self, *, classes, alpha = 1.0, rho = 1.0, dropout = 0.001, img_size = (224,224)):
        self._alpha = alpha
        self._rho = rho
        self._num_classes = classes
        self.model = None
        self._dropout = dropout
        self.img_size = img_size

    def __Standard_Conv(self):
        return models.Sequential([
            Conv2D(filters= 32, kernel_size=(3,3), strides= (2,2), padding= 'valid'),
            BatchNormalization(),
            ReLU(max_value= 6)
        ])

    def __Depthwise_Conv(self, strides, padding):
        return models.Sequential([
            DepthwiseConv2D(kernel_size= (3,3), strides= strides, padding= padding),
            BatchNormalization(),
            ReLU(max_value= 6)
        ])

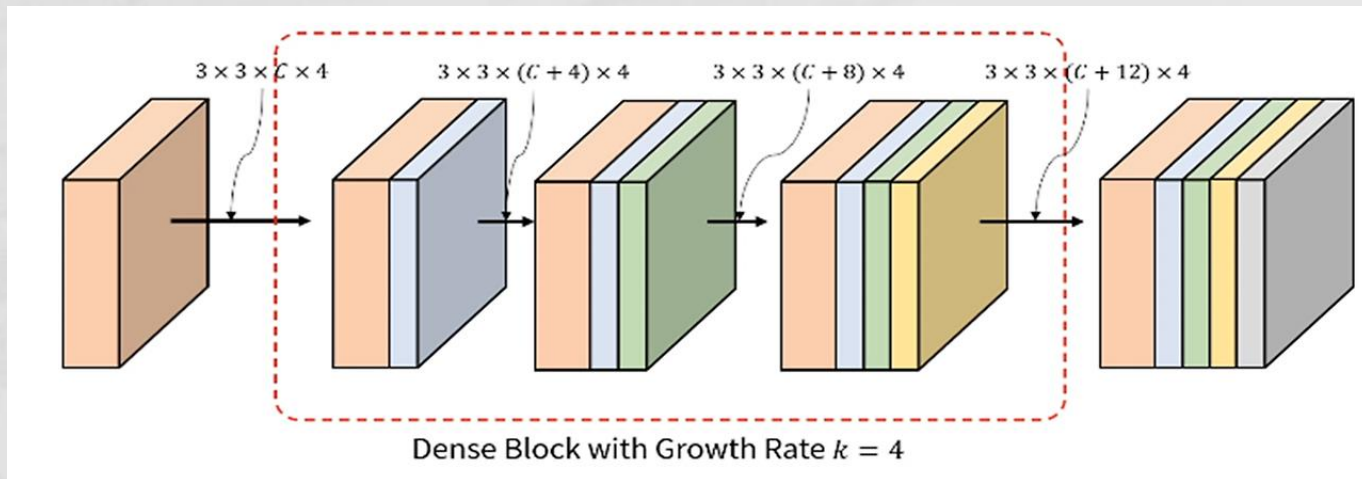
    def __Pointwise_Conv(self, filters):
        return models.Sequential([
            Conv2D(filters= int(filters * self._alpha), kernel_size= (1,1), strides= 1),
            BatchNormalization(),
            ReLU(max_value= 6)
        ])

    def __Depthwise_Separable_Conv(self, *, strides_depthwise, padding_depthwise, filters_pointwise):
        return models.Sequential([
            self.__Depthwise_Conv(strides= strides_depthwise, padding= padding_depthwise),
            self.__Pointwise_Conv(filters= filters_pointwise)
        ])
```

DenseNet

■ 개요

- ResNet의 문제점을 보완. 이 페이퍼에 따르면 identity mapping layer에서 $x + F(x)$ 로 입력값과 출력값을 단순히 더해서 출력하는 것은 정보의 흐름을 방해할 수 있다고 주장함.
- DenseNet 이 값을 더한게 아니라 depth-wise로 그냥 이어붙이기 한 것.
- 두 번째 그림의 빨간색 부분의 depth = C , 파란색 부분 depth = 4 . 따라서 이 활성화맵에 적용되는 필터의 depth = $C + 4$



DenseNet

```
class DenseNet(nn.Module):
    r"""Densenet-BC model class, based on
    ^"Densely Connected Convolutional Networks" <https://arxiv.org/pdf/1608.06993.pdf>`_

    Args:
        growth_rate (int) - how many filters to add each layer (`k` in paper)
        block_config (list of 4 ints) - how many layers in each pooling block
        bn_size (int) - multiplicative factor for number of bottle neck layers
            (i.e. bn_size * k features in the bottleneck layer)
        drop_rate (float) - dropout rate after each dense layer
        num_classes (int) - number of classification classes
        memory_efficient (bool) - If True, uses checkpointing. Much more memory efficient,
            but slower. Default: *False*. See ``paper" <https://arxiv.org/pdf/1707.06990.pdf>`_
    """
    def __init__(self, growth_rate=32, block_config=(6, 12, 24, 16), bn_size=4, stem_type='',
                 num_classes=1000, in_chans=3, global_pool='avg',
                 norm_layer=BatchNormAct2d, aa_layer=None, drop_rate=0, memory_efficient=False,
                 aa_stem_only=True):
        self.num_classes = num_classes
        self.drop_rate = drop_rate
        super(DenseNet, self).__init__()

        # Stem
        deep_stem = 'deep' in stem_type # 3x3 deep stem
        num_init_features = growth_rate * 2
        if aa_layer is None:
            stem_pool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        else:
            stem_pool = nn.Sequential(*[
                nn.MaxPool2d(kernel_size=3, stride=1, padding=1),
                aa_layer(channels=num_init_features, stride=2)])
        if deep_stem:
            stem_chs_1 = stem_chs_2 = growth_rate
            if 'tiered' in stem_type:
                stem_chs_1 = 3 * (growth_rate // 4)
                stem_chs_2 = num_init_features if 'narrow' in stem_type else 6 * (growth_rate // 4)
            self.features = nn.Sequential(OrderedDict([
                ('conv0', nn.Conv2d(in_chans, stem_chs_1, 3, stride=2, padding=1, bias=False)),
                ('norm0', norm_layer(stem_chs_1)),
                ('conv1', nn.Conv2d(stem_chs_1, stem_chs_2, 3, stride=1, padding=1, bias=False)),
                ('norm1', norm_layer(stem_chs_2)),
                ('conv2', nn.Conv2d(stem_chs_2, num_init_features, 3, stride=1, padding=1,
```

Efficient Net

■ 개요

- 2019년 구글에서 제안한 비교적 최신 기법이다. 사전학습 모형으로 많이 사용되지는 않음
- compound scaling method를 사용함. Width / Depth / Resolution 등을 고려해야 하는데 컴퓨터의 성능에 따라서 적절하게 선택해야 한다. 따라서 사용자의 환경에 따라서 다름

EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

Mingxing Tan¹ Quoc V. Le¹

Abstract

Convolutional Neural Networks (ConvNets) are commonly developed at a fixed resource budget, and then scaled up for better accuracy if more resources are available. In this paper, we systematically study model scaling and identify that carefully balancing network depth, width, and resolution can lead to better performance. Based on this observation, we propose a new scaling method that uniformly scales all dimensions of depth/width/resolution using a simple yet highly effective *compound coefficient*. We demonstrate the effectiveness of this method on scaling up MobileNets and ResNet.

To go even further, we use neural architecture search to design a new baseline network and scale it up to obtain a family of models, called *EfficientNets*, which achieve much better accuracy and efficiency than previous ConvNets. In particular, our EfficientNet-B7 achieves state-of-the-art 84.4% top-1 / 97.1% top-5 accuracy on ImageNet, while

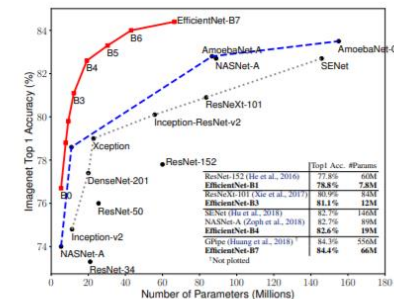


Figure 1. Model Size vs. ImageNet Accuracy. All numbers are for single-crop, single-model. Our EfficientNets significantly outperform other ConvNets. In particular, EfficientNet-B7 achieves new state-of-the-art 84.4% top-1 accuracy but being 8.4x smaller and 6.1x faster than GPipe. EfficientNet-B1 is 7.6x smaller and 5.7x faster than ResNet-152. Details are in Table 2 and 4.

Efficient Net

```
class EfficientNet(nn.Module):
    def __init__(self, num_classes=10, width_coef=1., depth_coef=1.,
scale=1., dropout=0.2, se_scale=4, stochastic_depth=False, p=0.5):
        super().__init__()
        channels = [32, 16, 24, 40, 80, 112, 192, 320, 1280]
        repeats = [1, 2, 2, 3, 3, 4, 1]
        strides = [1, 2, 2, 2, 1, 2, 1]
        kernel_size = [3, 3, 5, 3, 5, 5, 3]
        depth = depth_coef
        width = width_coef

        channels = [int(x*width) for x in channels]
        repeats = [int(x*depth) for x in repeats]

        # stochastic depth
        if stochastic_depth:
            self.p = p
            self.step = (1 - 0.5) / (sum(repeats) - 1)
        else:
            self.p = 1
            self.step = 0

        # efficient net
        self.upsample = nn.Upsample(scale_factor=scale, mode='bilinear',
align_corners=False)

        self.stage1 = nn.Sequential(
            nn.Conv2d(3, channels[0], 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(channels[0], momentum=0.99, eps=1e-3)
        )
```

정리

정리

- CNN 모델 종류